

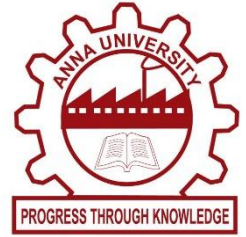


**V.S.B. ENGINEERING COLLEGE**  
**(An Autonomous Institution)**

**(Anna University Recognized Research Institute)**

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai

NBA Accredited Courses, Accredited by NAAC



**"FSLAKWS – Few Shot Language-Agnostic Key Word Spotting System"**

**A MINI PROJECT REPORT**

**submitted by**

**VAISNAVA S**

**922524148116**

**in partial fulfilment for the award of the degree of**

**BACHELOR OF ENGINEERING**

**in**

**COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL  
INTELLIGENCE AND MACHINE LEARNING)**

**V.S.B. ENGINEERING COLLEGE, KARUR**

**(AN AUTONOMOUS INSTITUTION)**

**AFFILIATED TO ANNA UNIVERSITY,**

**CHENNAI 600 025**

**JULY 2026**

**V.S.B. ENGINEERING COLLEGE, KARUR  
(AN AUTONOMOUS INSTITUTION)  
AFFILATED TO ANNA UNIVERSITY,  
CHENNAI 600 025**

**BONAFIDE CERTIFICATE**

Certified that this project report **“FSLAKWS – Few Shot Language-Agnostic Keyword Spotting System”** is the Bonafide work of **“VAISNAVA S (922524148116)”** who carried out the mini project work under my supervision.

**HEAD OF THE DEPARTMENT**

Mrs. D. Maalini, M.E.  
Assistant Professor and Head  
Department of Computer Science and  
Engineering (Artificial Intelligence and  
Machine Learning)  
V.S.B. Engineering College  
Karur - 639111

**SUPERVISOR**

Dr. B. Prasanth, M.E., PhD  
Assistant Professor  
Department of Computer Science and  
Engineering (Artificial Intelligence  
and Machine Learning)  
V.S.B. Engineering College  
Karur - 639111

## ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to the management of **V.S.B. Engineering College, Karudayampalayam, Karur**, for providing the necessary infrastructure, laboratory facilities, and academic environment that made the successful completion of this project possible.

I express my heartfelt thanks to **Dr. V. S. Balsamy, Chairman of V.S.B Engineering College**, for his valuable patronage and encouragement throughout our academic journey.

I am deeply grateful to **Dr. C. Vennila**, Principal, and **Dr. T. S. Kiruba Shankar**, Vice Principal, of V.S.B Engineering College, for their constant support and motivation in carrying out this project work.

I am also grateful to the Head of the Department, **Mrs. D. Maalini, M.E** and the faculty members of the Department of **Computer Science and Engineering (Artificial Intelligence and Machine Learning)** for their guidance and encouragement, and for fostering an environment that allowed me to undertake and complete this project independently during the semester break.

I take this opportunity to express my sincere thanks to my project guide, **Dr. B. Prasanth, M.E., PhD Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)** for the valuable guidance, continuous encouragement, and constructive feedback provided throughout the course of this project, titled "**FSLAKWS — Few-Shot Language-Agnostic Keyword Spotting System**". Their support was instrumental in helping me approach and resolve the technical challenges encountered during development.

This project was conceived, designed, and implemented entirely by me, and it reflects my individual effort to apply the concepts learned during my coursework to a practical, real-world problem.

Finally, I extend my sincere thanks to my family and friends for their constant encouragement, patience, and moral support throughout this endeavour.

## ABSTRACT

**FSLAKWS (Few-Shot Language-Agnostic Keyword Spotting System)** is an intelligent audio search tool that locates the occurrences of a specific spoken word or phrase within a longer audio recording, functioning much like a "Ctrl + F" for audio. Unlike conventional keyword spotting systems that require large labelled datasets and are trained for a fixed vocabulary in a specific language, FSLAKWS uses a few-shot learning approach: the user provides only five sample recordings of the target keyword, and the system builds an acoustic template from these samples without needing prior training on that word. This makes the system language-agnostic, since it relies on acoustic similarity rather than transcribed vocabulary.

The system extracts Mel-Frequency Cepstral Coefficient (MFCC) features from the keyword samples and the target audio, then applies Dynamic Time Warping (DTW) to measure the similarity between the keyword template, Speech Recognition API for keyword verification and sliding windows across the target audio. Matches below a similarity threshold are reported with their timestamps. The system also verifies, using speech recognition, that all five keyword samples correspond to the same word before proceeding, returning an error if they do not.

FSLAKWS is implemented as a three-tier web application: a React.js frontend for user interaction, a Spring Boot backend for authentication and data management, and a Python-based machine learning service for audio processing. A MySQL database stores user accounts, enrolled keywords, and search results. The system offers a practical, low-resource solution for keyword spotting tasks such as searching lecture recordings, call transcripts, or field recordings for a specific word without requiring large-scale model training.

## TABLE OF CONTENTS

| Section          | Contents                             | Page No.  |
|------------------|--------------------------------------|-----------|
|                  | Cover Page                           | 1         |
|                  | Bonafide                             | 2         |
|                  | Acknowledgement                      | 3         |
|                  | Abstract                             | 4         |
|                  | Table of Contents                    | 5         |
| <b>Chapter 1</b> | <b>Introduction</b>                  | <b>7</b>  |
| 1.1              | Background                           | 7         |
| 1.2              | Objectives                           | 7         |
| 1.3              | Scope of the Project                 | 8         |
| <b>Chapter 2</b> | <b>Literature Survey</b>             | <b>9</b>  |
| <b>Chapter 3</b> | <b>System Design and Methodology</b> | <b>11</b> |
| 3.1              | System Architecture                  | 11        |
| 3.2              | Existing System                      | 11        |
| 3.3              | Proposed System                      | 12        |
| 3.4              | Block Diagram                        | 14        |
| 3.5              | Flowchart                            | 15        |
| 3.6              | Hardware Requirements                | 16        |
| 3.7              | Software Requirements                | 16        |
| 3.8              | Development Process                  | 16        |
| <b>Chapter 4</b> | <b>Implementation</b>                | <b>18</b> |
| 4.1              | Project Development                  | 18        |
| 4.2              | System Components and Architecture   | 18        |

|                  |                               |           |
|------------------|-------------------------------|-----------|
| 4.3              | Operational Workflows         | 22        |
| 4.4              | Software Coding Demonstration | 23        |
| 4.5              | Working Principle             | 26        |
| 4.6              | Implementation Steps          | 27        |
| <b>Chapter 5</b> | <b>Results and Discussion</b> | <b>28</b> |
| 5.1              | Output                        | 28        |
| 5.2              | Testing                       | 31        |
| 5.3              | Observations                  | 32        |
| 5.4              | Advantages                    | 32        |
| 5.5              | Future Scope                  | 33        |
| <b>Chapter 6</b> | <b>Conclusion</b>             | <b>34</b> |
|                  | <b>References</b>             | <b>35</b> |

# 1. INTRODUCTION

## 1.1 Background

Searching for a specific spoken word within an audio recording is a common but difficult task. Unlike text, audio has no built-in search function — finding where a word occurs in a one-hour lecture, a call recording, or a field interview traditionally requires either manually listening through the entire file or using a full automatic speech recognition (ASR) system to transcribe the audio and then perform a text search. ASR-based approaches require large, labelled training datasets for each language and often perform poorly on low-resource languages, dialects, or unfamiliar vocabulary such as names or technical terms.

**Keyword spotting (KWS) systems** address this by detecting the presence of a specific word directly from audio, without full transcription. However, most existing KWS systems are trained for a fixed set of keywords in a specific language and cannot generalize to a new word without retraining. This limits their usefulness in scenarios where the target keyword is unknown in advance or changes from search to search.

FSLAKWS is designed to overcome this limitation using a few-shot, language-agnostic approach. Instead of training a model on a large labelled dataset, the user simply provides five example recordings of the keyword. The system builds an acoustic template from these examples and searches for acoustically similar segments in the target audio. Since the method relies on acoustic feature comparison (MFCC + DTW) rather than a trained language model, it works across languages and is not limited to a fixed vocabulary.

## 1.2 Objectives

The main objectives of the proposed project are:

- To design a keyword spotting system that requires only a few sample recordings of a keyword instead of a large labelled dataset.
- To make the system language-agnostic by relying on acoustic feature similarity rather than transcription-based matching.
- To verify that all provided keyword samples correspond to the same spoken word before performing a search.

- To locate and report the timestamps of keyword occurrences within a longer target audio recording.
- To provide both file upload and live microphone recording options for keyword input.
- To develop a complete web application with user authentication, keyword enrolment, and audio search functionality.
- To build a low-cost, practical solution deployable without specialized hardware.

### **1.3 Scope of the Project**

The scope of this project is to develop a web-based system that allows a registered user to enrol a keyword using five audio samples (uploaded or recorded), upload a target audio file, and receive the timestamps at which the keyword occurs in that target audio. The system verifies keyword consistency across the five samples using speech recognition before performing the search.

project can be applied to use cases such as searching lecture or meeting recordings for a specific term, scanning call centre recordings for a keyword, or searching field-recorded interviews in local languages where standard ASR support is limited. The prototype is built using freely available open-source libraries and does not require specialized hardware.

## 2. LITERATURE SURVEY

Keyword spotting has been an active research area for several decades, driven by applications ranging from voice assistants ("wake word" detection) to audio indexing and retrieval. Traditional KWS systems are typically built using Hidden Markov Models (HMMs) or, more recently, deep neural networks trained on large labelled corpora for a fixed set of target words. While effective for common wake words such as "Hey Siri" or "OK Google," these systems require substantial retraining effort to support a new keyword, making them impractical for open-vocabulary or user-defined search tasks.

Query-by-example (QbE) keyword spotting emerged as an alternative that avoids large-scale retraining. In QbE-KWS, the user provides one or more example recordings of the keyword, and the system searches for acoustically similar patterns in the target audio using distance-based template matching. Dynamic Time Warping (DTW) has been widely used in QbE-KWS research because it can align two audio sequences of different lengths and speaking rates, which is essential since the keyword sample and its occurrence in the target audio are rarely spoken at exactly the same speed.

Mel-Frequency Cepstral Coefficients (MFCCs) remain a standard acoustic feature representation for speech and audio comparison tasks, since they approximate the human auditory system's response and are relatively robust to speaker and recording variation. Combining MFCC feature extraction with DTW-based distance scoring forms a well-established baseline approach for template-based audio matching, used in earlier QbE-KWS literature before the widespread adoption of deep embeddings.

Few-shot learning approaches have more recently been applied to keyword spotting, where a small number of examples (typically five or fewer) are used to construct a keyword representation, often through neural embeddings trained via metric learning. These approaches generally outperform simple MFCC+DTW baselines in accuracy but require pretrained embedding models and larger computational resources, which may not be readily available for a low-resource undergraduate project setting.

Language-agnostic keyword spotting is particularly relevant for regions with many low-resource languages and dialects, where labeled ASR training data is scarce or unavailable. Since acoustic template-matching methods like MFCC+DTW do not rely on a language model or a fixed vocabulary, they can, in

principle, be applied to any spoken language as long as example recordings of the keyword are available.

FSLAKWS builds on this line of work by combining an MFCC+DTW-based query-by-example approach with a five-sample few-shot enrolment step and a speech-recognition-based consistency check to ensure the enrolled samples represent the same word, addressing a practical gap between purely rule-based QbE systems and the ASR-based verification of keyword identity.

## **3. SYSTEM DESIGN AND METHODOLOGY**

### **3.1 System Architecture**

FSLAKWS is designed as a three-tier web application consisting of a frontend, a backend, and a machine learning service. The React.js frontend handles user interaction, including sign-in, registration, keyword enrolment (uploading or recording five audio samples), and target audio upload. The Spring Boot backend manages user authentication, stores enrolled keywords and search results, and coordinates requests between the frontend and the machine learning service. The Python-based machine learning service performs the core audio processing: feature extraction, keyword consistency verification, template construction, and DTW-based search. A MySQL database persists user accounts, keyword metadata, and search history.

### **3.2 Existing System**

#### **3.2.1 Overview of Current Methodologies**

Existing systems for locating specific words or phrases in speech primarily rely on two main approaches: Automatic Speech Recognition (ASR) engines (such as Whisper, Google Cloud Speech-to-Text, or Kaldi) or Deep Learning-based Keyword Spotting (KWS) models (such as ResNet, Convolutional Neural Networks (CNNs), or Recurrent Neural Networks (RNNs)). While ASR systems transcribe continuous speech into text, traditional KWS systems are designed to detect a predefined set of wake words (like "Hey Siri" or "OK Google").

#### **3.2.2 Key Limitations and Disadvantages of the Existing System**

##### **1. Heavy Data and Training Requirements**

Modern deep learning-based Keyword Spotting models are data-hungry. To train a model to recognize a new custom keyword with high accuracy, it requires a massive dataset containing thousands of audio samples of that specific keyword. These samples must be spoken by diverse speakers with different accents, pitches, and background noise profiles. Collecting, labeling, and preprocessing such a dataset is extremely time-consuming, expensive, and practically impossible for individual users or personalized search tasks.

## 2. Language Dependency and Vocabulary Constraints

Standard speech recognition systems are highly language-dependent. They are bound to predefined acoustic models and large language vocabularies trained on dominant global languages (e.g., English, Spanish, Mandarin). When faced with low-resource regional languages (such as Tamil, Telugu, Hindi, or Malayalam) or local dialects, these systems perform poorly. Furthermore, if a user wants to search for a custom keyword—such as a specific name, local slang, or a technical term that is not part of the ASR engine's pre-trained vocabulary—the system either misinterprets the word or fails to detect it entirely.

## 3. Computational and Infrastructure Overhead

ASR-based search systems require transcribing the *entire* audio file into text before performing a text-based search queries. Transcribing hours of audio is computationally expensive, slow, and requires heavy infrastructure (like high-end GPUs). If a user only needs to check whether a specific keyword was spoken, transcribing the entire file introduces unnecessary processing bottlenecks and latency.

## 4. Limitations of Manual Audio Search

When automated speech models are unavailable or fail, users must resort to manual keyword search. This involves playing a long audio recording and listening continuously to spot the target word. Manual search is highly inefficient, exhausting, prone to human error due to cognitive fatigue, and takes a significant amount of time (often taking longer than the actual duration of the audio clip).

## 3.3 Proposed System

### 3.3.1 Technical Concept and Solution

The proposed system, Few-Shot Language-Agnostic Keyword Spotting (FSLAKWS), introduces an innovative framework that solves the data scarcity and language dependency bottlenecks of traditional speech recognition. The core objective of the proposed system is to learn and detect a custom keyword from just 5 user-provided audio samples, operating independently of the spoken language. The system is built on a multi-tier architecture consisting of a React JS (Vite) frontend, a Spring Boot enterprise backend, a MySQL relational database, and a Python Flask Machine Learning service.

### **3.3.2 Functional Architecture**

#### **1. Audio Preprocessing and Feature Extraction**

When the user records (via the browser's WebRTC API) or uploads 5 samples of a keyword and a target audio file, the system standardizes the audio properties. The files are converted into single-channel (mono), 16kHz sample rate WAV files using pydub. The system then extracts Mel-Frequency Cepstral Coefficients (MFCCs) using the librosa library. MFCCs represent the short-term power spectrum of the audio, capturing the physical and vocal characteristics of the spoken word while filtering out irrelevant background noise.

#### **2. Few-Shot Keyword Consistency Verification**

To prevent users from uploading mismatching or corrupted samples (e.g., uploading 3 files saying "hello" and 2 files saying "world"), the Flask ML service integrates a verification check using Google Speech-to-Text. The system transcribes the 5 audio samples and cross-references the text results. If the transcripts do not match, the enrollment is rejected. Once verified, the keyword is registered.

#### **3. Acoustic Template Consolidation**

Instead of matching the target audio against all 5 samples individually (which increases processing time and error rates), the proposed system synthesizes a single consolidated acoustic template. It calculates the median frame length of the 5 sample MFCC sequences, resamples the sequences to this target length using linear interpolation along the time axis, and averages them. This temporal averaging cancels out random speech variations, creating a high-quality, noise-resistant representation of the keyword.

#### **4. Fast Dynamic Time Warping (FastDTW) Search Engine**

To search the target audio, the system employs a sliding-window search. The window width is set to match the duration of the consolidated keyword template. As the window slides across the target audio's MFCC sequence, it calculates the alignment distance using Fast Dynamic Time Warping (FastDTW) with Euclidean distance metrics. DTW is essential because it accounts for temporal distortions—meaning it successfully matches the keyword

even if the speaker in the target audio speaks much faster or slower than the enrolment samples.

## 5. Adaptive Thresholding and Timestamp Extraction

The system calculates matching thresholds dynamically based on the statistical properties (mean and standard deviation) of the DTW distance scores across the audio file. Windows with distances falling below this adaptive threshold are marked as matches. Overlapping or closely adjacent matches (within a 0.5-second buffer) are merged, retaining only the coordinate with the lowest alignment distance. The resulting timestamps are returned to the Spring Boot backend, persisted in the MySQL database, and rendered to the user on an interactive results dashboard equipped with an audio player.

### 3.4 Block Diagram

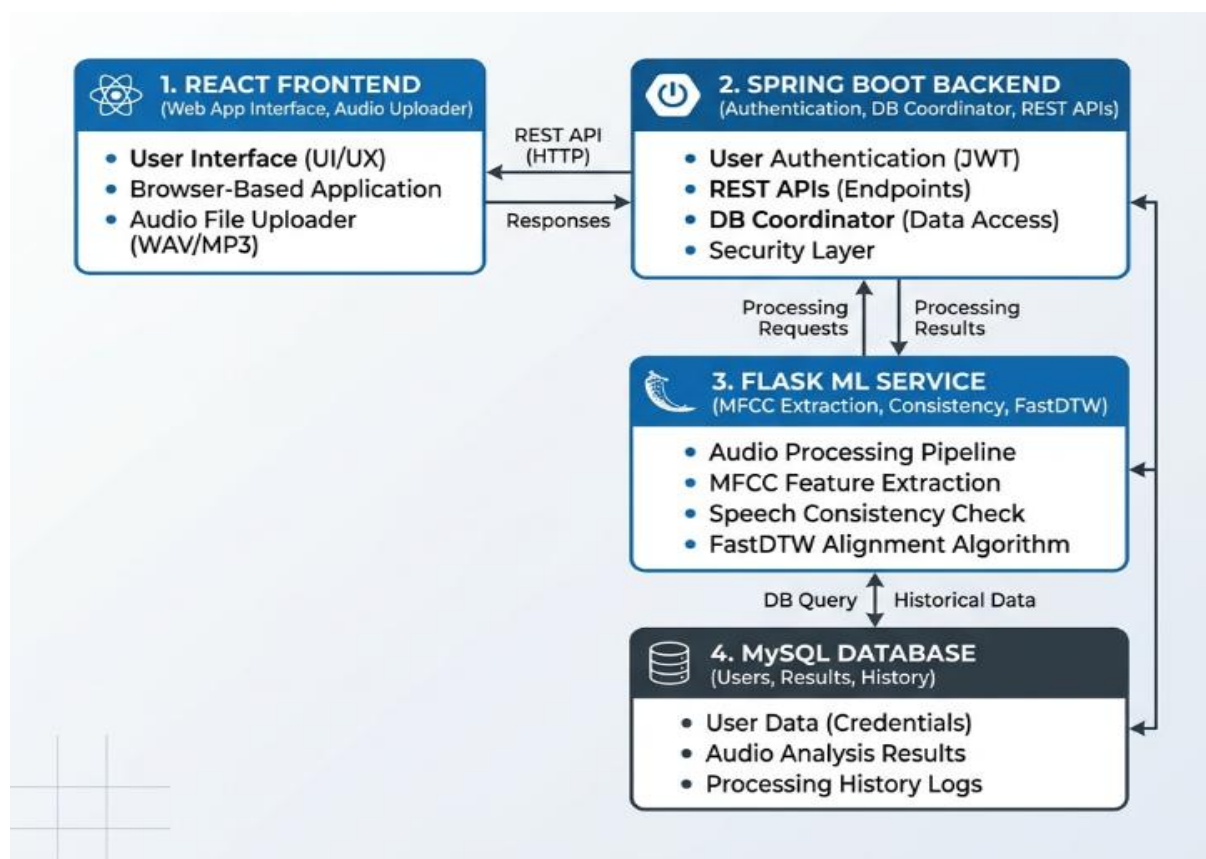


Figure 3. 1 Block diagram of FSLAKWS

## Working of the Block Diagram:

- The React frontend collects five keyword audio samples (upload or record) and one target audio file (upload only).
- The Spring Boot backend authenticates the user and forwards the audio data to the Python ML service.
- The ML service verifies that all five keyword samples are the same word using speech recognition, and returns an error if they are not.
- If verified, the ML service builds a consolidated keyword template and searches the target audio using MFCC + DTW.
- Results (timestamps) are returned to the backend, stored in MySQL, and displayed to the user via the frontend.

## 3.5 Flowchart

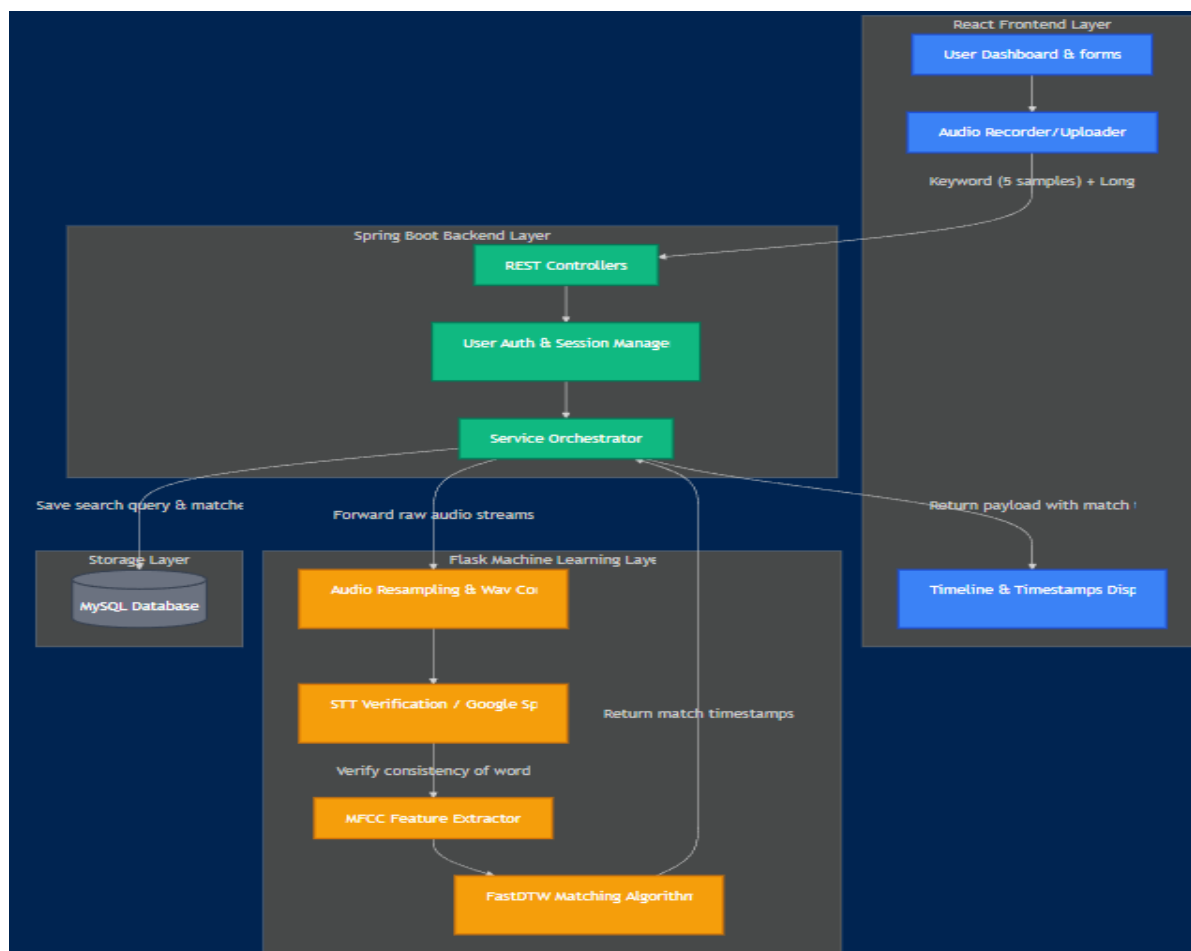


Figure 3. 2 Flow Chart for FSLAKWS

### 3.6 Hardware Requirements

*Table 3.1 Basic Hardware Requirements*

| S.No | Component           | Specification                                    |
|------|---------------------|--------------------------------------------------|
| 1    | Development machine | Intel i5 or equivalent, 8GB RAM minimum          |
| 2    | Microphone          | Built-in or external, for live keyword recording |
| 3    | Internet connection | Required for speech recognition API calls        |

### 3.7 Software Requirements

*Table 3.2 Software packages and its purpose*

| Software                 | Purpose                                           |
|--------------------------|---------------------------------------------------|
| React.js                 | Frontend user interface                           |
| Spring Boot (Java)       | Backend REST API, authentication, data management |
| Python 3.10              | Machine learning service                          |
| librosa                  | Audio feature extraction (MFCC)                   |
| SpeechRecognition, pydub | Keyword consistency verification                  |
| MySQL                    | Database for users, keywords, and results         |
| VS Code                  | Development environment                           |
| Git / GitHub             | Version control                                   |

### 3.8 Development Process

**Step 1 - Requirement Analysis:** The project objectives were identified, and the technology stack (React, Spring Boot, Python, MySQL) was selected based on the need for a full-stack web application with a dedicated audio processing service.

**Step 2 - System Design:** The block diagram, data flow, and API structure between the frontend, backend, and ML service were designed.

**Step 3 - ML Service Development:** The Python service was developed first to validate the core keyword spotting approach — audio input collection, keyword verification, template building, and DTW-based search — since this was the highest-risk component.

**Step 4 - Frontend Development:** The React frontend was built with sign-in, registration, keyword enrolment, and audio search pages.

**Step 5 - Backend Development:** The Spring Boot backend was developed to handle authentication and data persistence, and connected to the frontend and ML service.

**Step 6 - Testing:** The system was tested end-to-end: registering a user, enrolling a keyword, uploading a target audio file, and verifying that reported timestamps matched actual keyword occurrences.

**Step 7 - Result Validation:** The ML service was validated against real audio recordings, confirming that keyword occurrences were detected at approximately correct timestamps.

## 4. IMPLEMENTATION

### 4.1 Project Development

FSLAKWS was developed by first building and validating the Python machine learning service, since accurate keyword detection was the highest-risk and most critical component of the project. The service accepts five keyword audio samples (via file upload or live microphone recording) and one target audio file (upload only). It uses the SpeechRecognition library to transcribe each keyword sample and confirms that all five samples correspond to the same word; if they do not match, the system returns an error rather than proceeding. Once verified, MFCC features are extracted from all five samples and time-aligned to build a single consolidated keyword template. The target audio is then scanned using a sliding window with DTW-based distance scoring, and windows below a similarity threshold are reported as keyword matches, with duplicate nearby matches merged.

The React frontend was then developed to provide sign-in and registration pages, a keyword enrolment page supporting both upload and recording, and an audio search page for the target file. The Spring Boot backend was built to handle user authentication and persist enrolled keywords and search results to a MySQL database, connecting the frontend to the Python ML service via REST API calls.

### 4.2 System Components and Architecture

#### 4.2.1. Web Frontend (React / Vite)

- **Technologies:** React, Axios, Web Audio API (MediaRecorder).
- **Key Responsibilities:**
  - **Audio Recording:** Captures 5 microphone inputs using the Web Audio API and saves them locally as webm blobs.
  - **Enrolment Interface:** Collects keyword samples and names, sending them as multipart/form-data to the backend.
  - **Search Console:** Allows selecting a registered keyword and uploading a target audio file.

- **Dashboard & History Panels:** Renders previous keyword matches and historical search logs.

#### 4.2.2. Backend Orchestration (Spring Boot)

- **Technologies:** Java 21, Spring Boot, Spring Data JPA, Hibernate, Maven.
- **Key Responsibilities:**
  - **REST Controllers:** Exposes endpoints for authentication (AuthController), keyword management (KeywordSampleController), target audio (TargetAudioController), and searches (SearchController).
  - **Storage Management:** Saves the physical audio files to a disk directory structured by user ID.
  - **ML Client Dispatcher:** Uses Spring's RestTemplate to forward the 5 keyword wavs and the target audio to the Flask ML Service.
  - **Data Mapper:** Translates matching timestamps returned by the ML service into persistent database records (SearchResult and SearchHistory).

#### 4.2.3. Machine Learning Microservice (Python Flask)

- **Technologies:** Python 3, Flask, Librosa, FastDTW, SciPy, SpeechRecognition, PyDub.
- **Key Responsibilities:**
  - **Audio Processing:** Converts incoming .webm or other formats into mono 16kHz WAV format (via pydub).
  - **STT Validation:** Performs speech-to-text on the 5 keyword recordings using the Google Speech Recognition API to ensure all 5 samples contain the same spoken word. If transcripts do not match, the enrolment fails.
  - **Feature Extraction:** Extracts 13-dimensional Mel-Frequency Cepstral Coefficients (MFCCs) from each audio file.
  - **Consolidated Template Builder:** Normalizes and time-aligns (via linear interpolation) the 5 keyword MFCC matrices, calculating their mean to produce a single, robust template.

- **Keyword Spotting Matching:** Employs a sliding-window Dynamic Time Warping (DTW) algorithm comparing the consolidated template with the target audio.

#### 4.2.4. Database Storage (MySQL)

- **Driver:** MySQL Connector / Java (via Spring Boot Datasource).
- **Key Schemas:**
  - **Users:** Authentication parameters (id, name, email, password hash).

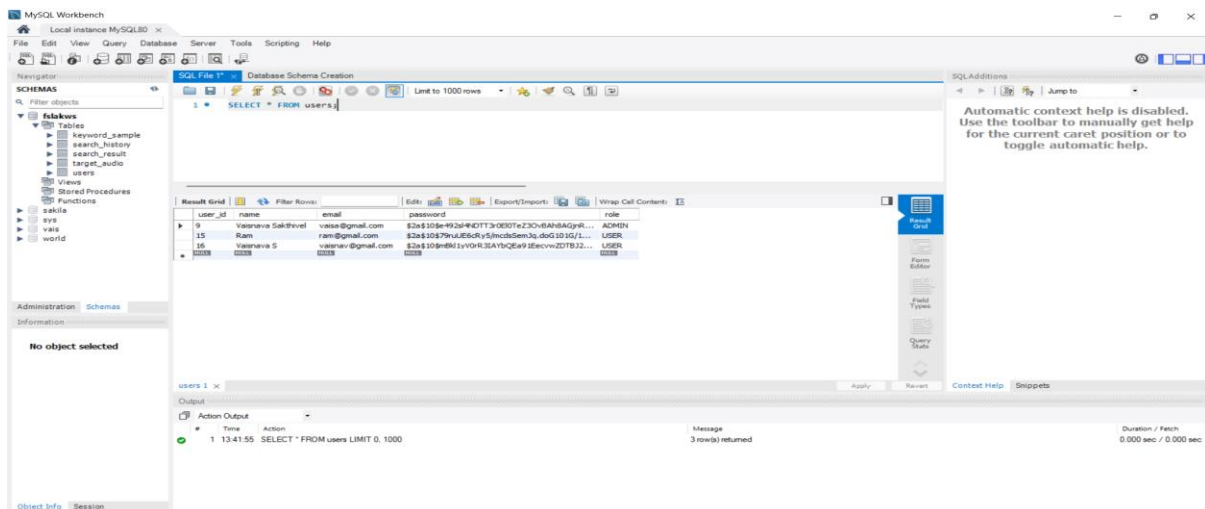


Figure 4. 1 User Table

- **Keyword\_Sample:** Metadata pointing to the 5 recorded sample paths on the server filesystem.

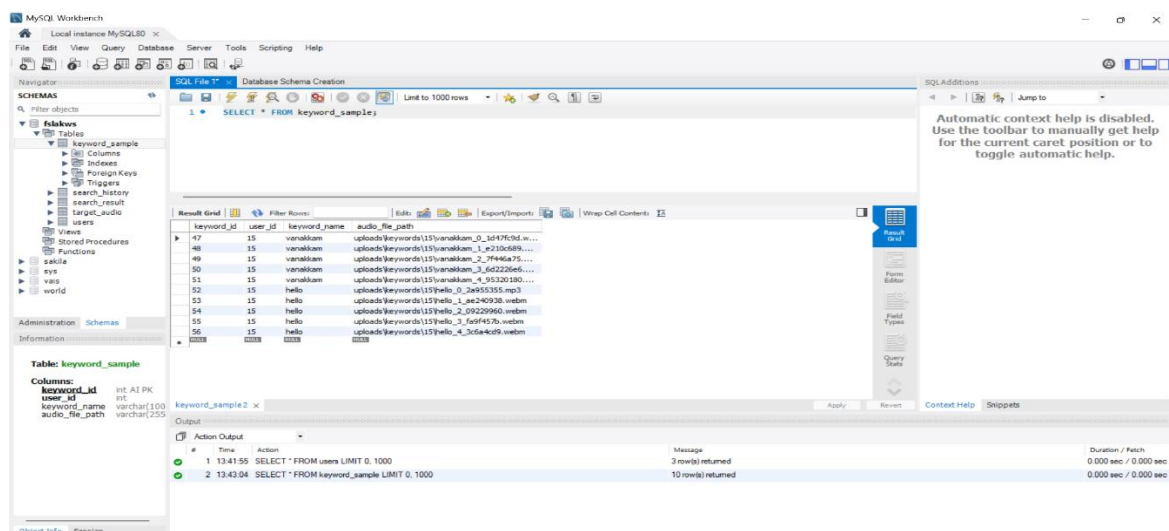


Figure 4. 2 Table for Keyword samples

- **Target\_Audio:** Record of target audio files uploaded for scanning.

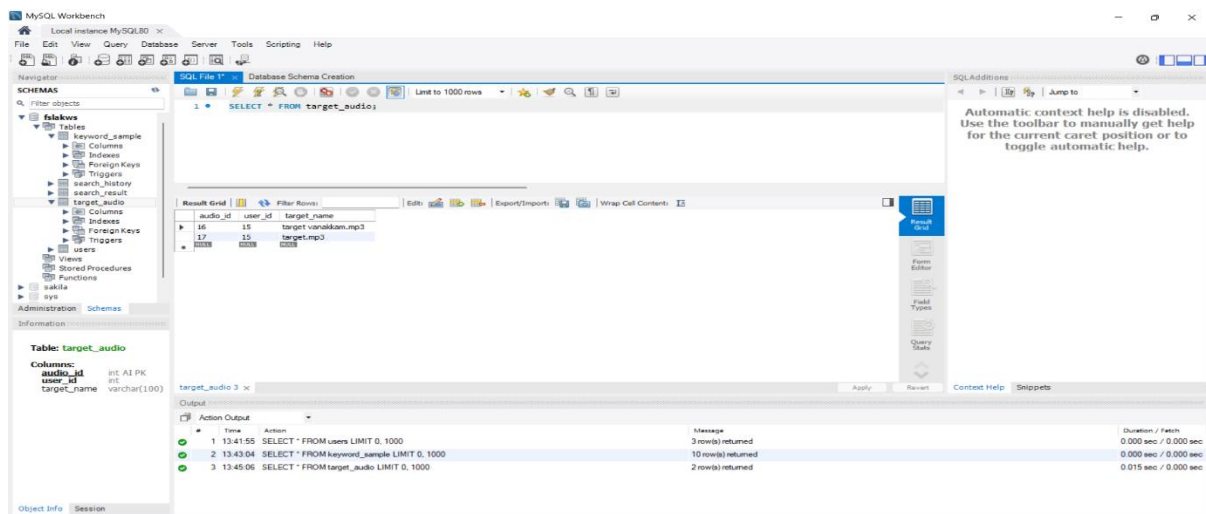


Figure 4. 3 Target Audio table

- **Search\_Result:** Stores match occurrences including start times, end times, and match scores.

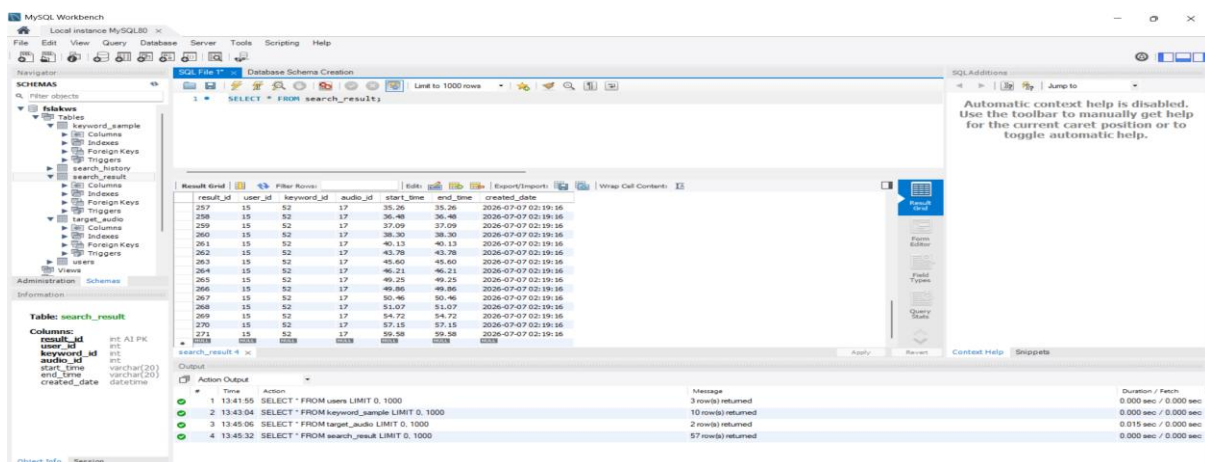


Figure 4. 4 Search result table

- **Search\_History:** Audit log of search tasks executed by users.

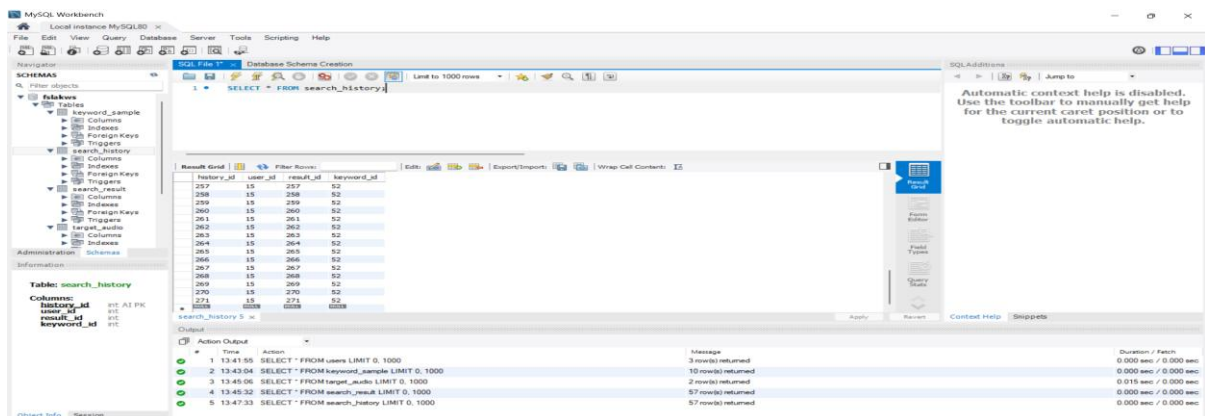


Figure 4. 5 Search History table

## 4.3 Operational Workflows

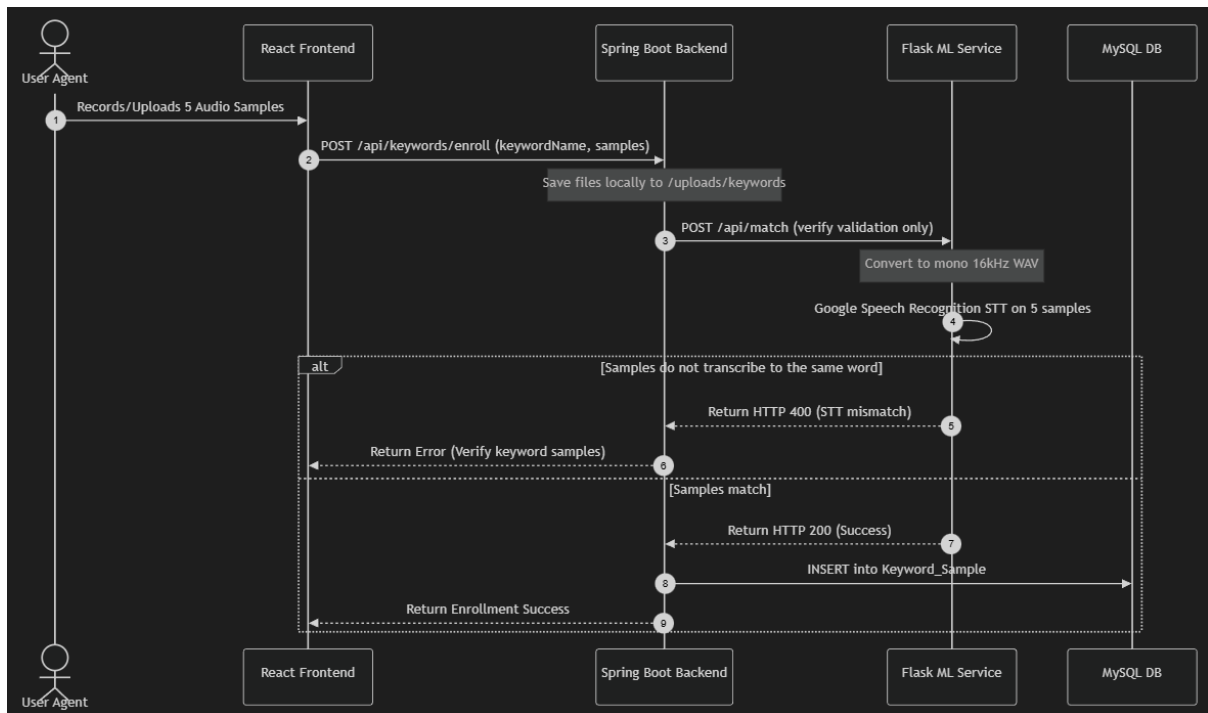


Figure 4. 6 Keyword enrollment flow

This flow details how the system validates and registers a keyword before it can be used for search tasks.

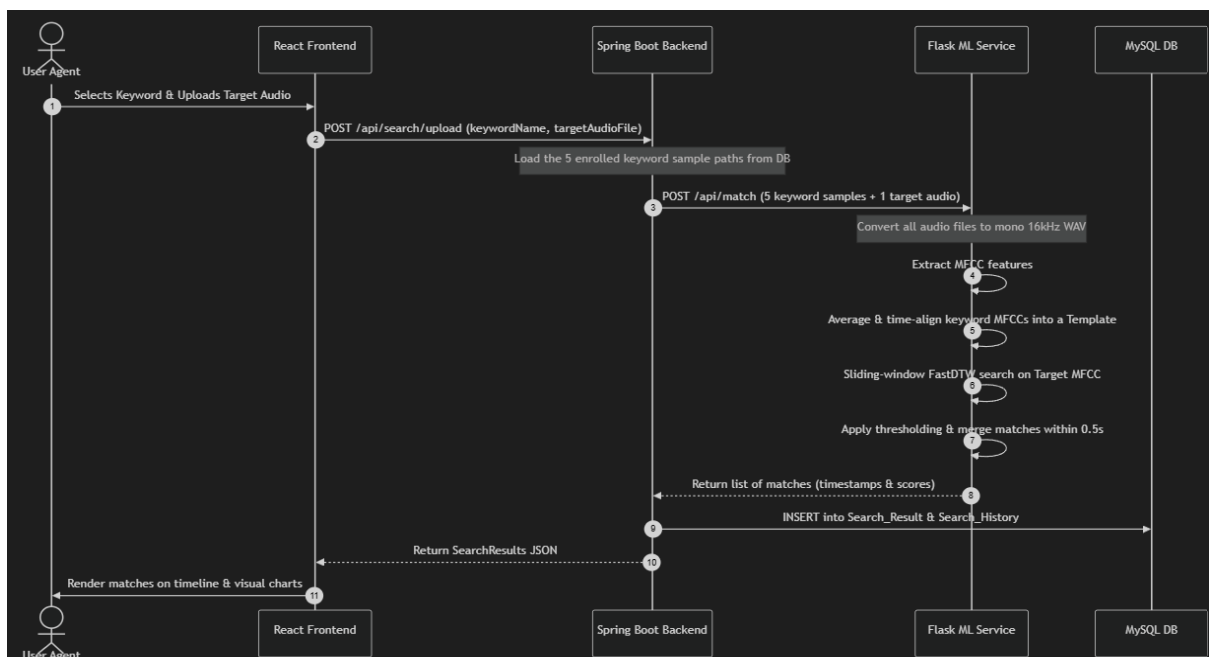


Figure 4. 7 Keyword Spotting Search Flow

This flow details what happens when a user scans a target audio file for an enrolled keyword.

## 4.4 Software Coding Demonstration

This section presents the critical source code snippets that implement the core functionalities of the FSLAKWS platform, demonstrating the end-to-end flow from browser recording to backend orchestration and DTW-based pattern matching.

### 4.4.1 Frontend: Web Audio Recording (React/JS)

- **Description:** This snippet demonstrates how the React application captures the user's microphone inputs natively using the browser's HTML5 Web Audio and MediaRecorder APIs. The recording is saved as a .webm audio blob and immediately added to the file upload payload.

#### CODE SNIPPET:

```
// Native Microphone Recording initialization and control
const startRecording = async (index) => {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio:
true });
    const mediaRecorder = new MediaRecorder(stream);
    mediaRecorders.current[index] = mediaRecorder;
    audioChunks.current[index] = [];

    mediaRecorder.ondataavailable = (event) => {
      if (event.data.size > 0) {
        audioChunks.current[index].push(event.data);
      }
    };
    mediaRecorder.onstop = () => {
      // Create audio blob from chunks
      const audioBlob = new Blob(audioChunks.current[index], { type:
'audio/webm' });
      const audioUrl = URL.createObjectURL(audioBlob);

      // Save URL to play back in browser
      const updatedUrls = [...audioUrls];
      updatedUrls[index] = audioUrl;
      setAudioUrls(updatedUrls);

      // Save as File instance for multipart upload
      const file = new File([audioBlob], `recorded_sample_${index +
1}.webm`, { type: 'audio/webm' });
      const updatedFiles = [...enrollFiles];
      updatedFiles[index] = file;
      setEnrollFiles(updatedFiles);
      // Stop mic hardware streams
      stream.getTracks().forEach(track => track.stop());
    };
  }
};
```

```

};
mediaRecorder.start();
const updatedStates = [...recordingStates];
updatedStates[index] = true;
setRecordingStates(updatedStates);
} catch (err) {
  console.error('Microphone access denied:', err);
  alert('Microphone access is required to record audio samples!');
}
};

```

#### 4.4.2 Backend: REST Client & ML Dispatcher (Spring Boot/Java)

- **Description:** This method coordinates the workflow: it saves the target audio on the server, collects the 5 previously enrolled keyword samples from the database, packs them into a multi-part form-data body, and sends a REST request to the Python Flask ML service using Spring's RestTemplate.

#### CODE SNIPPET:

```

// Save target audio locally and forward the files to Python ML
service
public List<SearchResult> uploadAndSearch(Integer userId, String
keywordName, MultipartFile targetAudioFile) throws IOException {
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new RuntimeException("User not found
with id: " + userId));

    // Get 5 enrolled keywords for templates
    List<KeywordSample> enrolledSamples =
keywordSampleRepository.findById(userId);
    List<KeywordSample> targetSamples = enrolledSamples.stream()
        .filter(sample ->
sample.getKeywordName().equalsIgnoreCase(keywordName))
        .limit(5)
        .toList();

    // 1. Save Target Audio to disk
    String uploadDir = "uploads" + File.separator + "targets" +
File.separator + userId;
    String filename = "target_" +
UUID.randomUUID().toString().substring(0, 8) + ".wav";
    Path targetPath = Paths.get(uploadDir, filename);
    Files.copy(targetAudioFile.getInputStream(), targetPath,
StandardCopyOption.REPLACE_EXISTING);

    // 2. Setup Multipart request to Python Flask
    RestTemplate restTemplate = new RestTemplate();
    String flaskUrl = "http://localhost:5000/api/match";

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.MULTIPART_FORM_DATA);

```

```

MultiValueMap<String, Object> body = new LinkedMultiValueMap<>();

// Add the 5 keyword recordings
for (KeywordSample sample : targetSamples) {
    body.add("keywordSamples", new FileSystemResource(new
File(sample.getAudioFilePath())));
}
// Add target audio file to scan
body.add("targetAudio", new
FileSystemResource(targetPath.toFile()));

HttpEntity<MultiValueMap<String, Object>> requestEntity = new
HttpEntity<>(body, headers);

// 3. Post to Flask API and parse response timestamps
List<SearchResult> searchResults = new ArrayList<>();
Map<String, Object> response =
restTemplate.postForObject(flaskUrl, requestEntity, Map.class);
if (response != null && response.containsKey("timestamps")) {
    List<Map<String, Object>> timestamps = (List<Map<String,
Object>>) response.get("timestamps");
    for (Map<String, Object> t : timestamps) {
        Number timeNum = (Number) t.get("time");
        String timestampStr = String.format("%.2f",
timeNum.doubleValue());

        // Persist matches and log to user search history
        SearchResult result = new SearchResult(user,
targetSamples.get(0), targetAudio, timestampStr, timestampStr);
        searchResults.add(searchResultRepository.save(result));
    }
}
return searchResults;
}

```

#### 4.4.3 Machine Learning: Acoustic Matching & DTW (Python/Flask)

- **Description:** This core algorithmic snippet shows how the Python service aligns the 5 keyword samples to build a single averaged template, runs a sliding window over the target audio's MFCC sequence, and applies Fast Dynamic Time Warping (FastDTW) to calculate similarity distances.

#### CODE SNIPPET:

```

def build_consolidated_template(keyword_mfccs):
    """Builds ONE template by averaging all 5 keyword MFCC sequences
(time-aligned)"""
    target_len = int(np.median([s.shape[0] for s in keyword_mfccs]))
    aligned = [resample_sequence(s, target_len) for s in
keyword_mfccs]
    consolidated = np.mean(aligned, axis=0)
    return consolidated

```

```

def find_keyword_matches(keyword_mfccs, target_mfcc, hop_length=512):
    # 1. Build a single consolidated template representation
    template = build_consolidated_template(keyword_mfccs)
    tpl_len = template.shape[0]
    n_frames = target_mfcc.shape[0]

    if n_frames <= tpl_len:
        return []

    # Slide window by step (1/4 of template length)
    step = max(1, tpl_len // 4)
    scores = []

    for start in range(0, n_frames - tpl_len, step):
        window = target_mfcc[start:start + tpl_len]
        # FastDTW calculates similarity aligning speech speed
        variations
        best_dist, _ = fastdtw(window, template, dist=euclidean)
        norm_dist = best_dist / max(1, window.shape[0])
        scores.append((start, norm_dist))

    # 2. Dynamic thresholding
    dists = np.array([s[1] for s in scores])
    threshold = np.mean(dists) - 0.5 * np.std(dists)

    matches = []
    for start, dist in scores:
        if dist <= threshold:
            time_sec = (start * hop_length) / SR
            matches.append({"time": round(time_sec, 2), "score":
round(float(dist), 3)})

    # 3. Merge overlapping matches within 0.5 seconds
    merged = []
    min_gap_sec = 0.5

    for m in matches:
        if merged and (m["time"] - merged[-1]["time"]) < min_gap_sec:
            if m["score"] < merged[-1]["score"]:
                merged[-1] = m
            else:
                merged.append(m)

    return merged

```

## 4.5 Working Principle

The system operates in two stages: keyword enrolment and audio search. During enrolment, the user provides five audio samples of the keyword. The system transcribes each sample using speech recognition and checks that all five recognized words match; if any sample differs, an error is shown and the process stops. If verified, the system extracts MFCC features from each sample and builds

a single consolidated template by time-aligning and averaging the five feature sequences.

During search, the user uploads a target audio file. The system extracts MFCC features from the target audio and compares the keyword template against sliding windows of the target using Dynamic Time Warping, which measures similarity between sequences even when speaking rate differs. Windows with a low DTW distance (high similarity) below a calculated threshold are reported as keyword matches, with their corresponding timestamps in the target audio. Nearby duplicate detections are merged to avoid reporting the same occurrence multiple times.

## **4.6 Implementation Steps**

1. Set up the React frontend project and required dependencies.
2. Set up the Spring Boot backend project and MySQL database.
3. Develop the Python ML service for audio feature extraction and search.
4. Implement keyword consistency verification using speech recognition.
5. Implement MFCC + DTW-based template matching.
6. Build the sign-in and registration pages.
7. Build the keyword enrolment page (upload/record).
8. Build the audio search page (upload target audio).
9. Connect frontend, backend, and ML service via REST APIs.
10. Test the complete system end-to-end with real audio recordings.

## 5. RESULTS AND DISCUSSION

### 5.1 Output

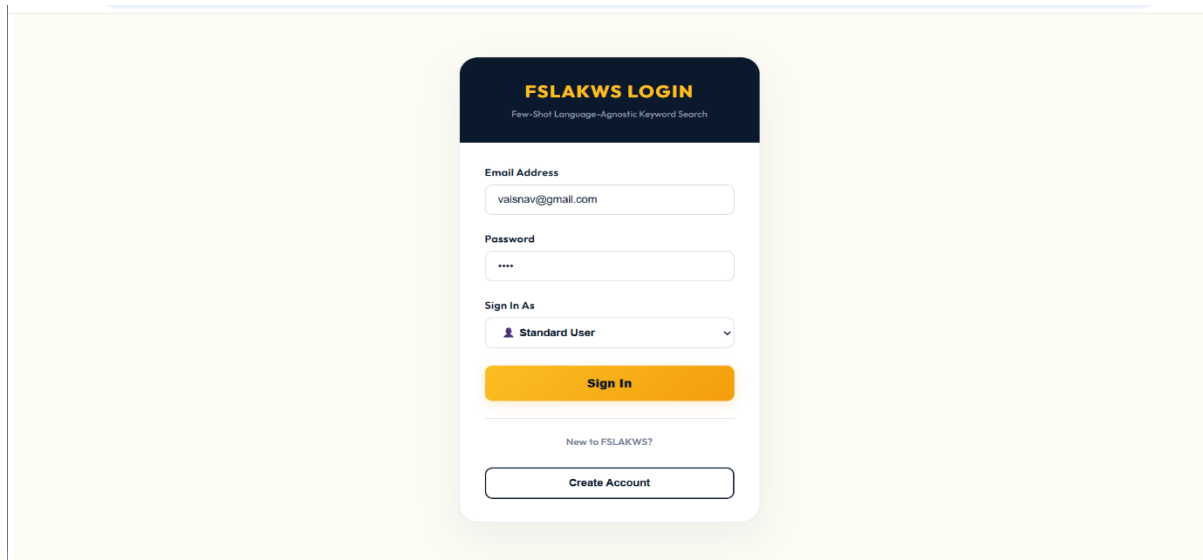
The developed system successfully verified keyword consistency across five audio samples and rejected samples that did not match. For matching samples, the system built a consolidated keyword template and searched a one-minute target audio file, returning multiple candidate timestamps. Manual verification of several reported timestamps confirmed that the keyword was indeed present at or near the reported times, indicating that the core detection approach works correctly on real audio. The following screenshots illustrate the major functionalities and workflow of the developed FSLAKWS system.

#### 5.1.1 Authentication pages

The authentication page of the FSLAKWS application, which enables both administrators and standard users to securely access the system based on their assigned roles.

##### 1. Login page

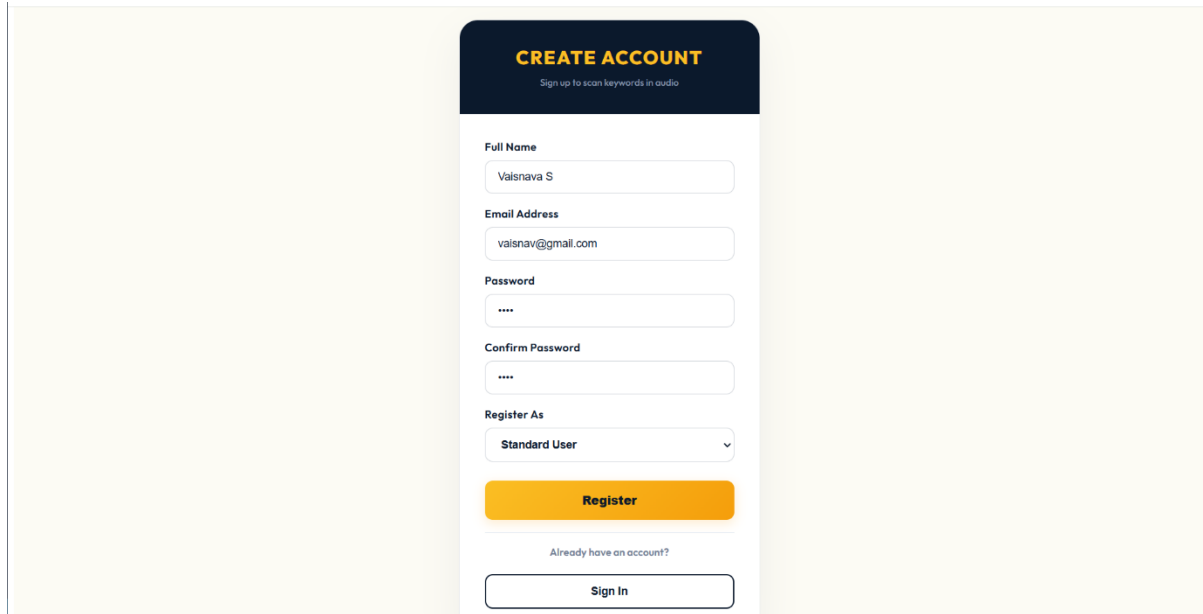
Registered users can securely access the system by entering their email address and password, selecting the appropriate user role, and clicking the "Sign In" button.



*Figure 5. 1 Login Page of FSLAKWS*

## 2. Registration page

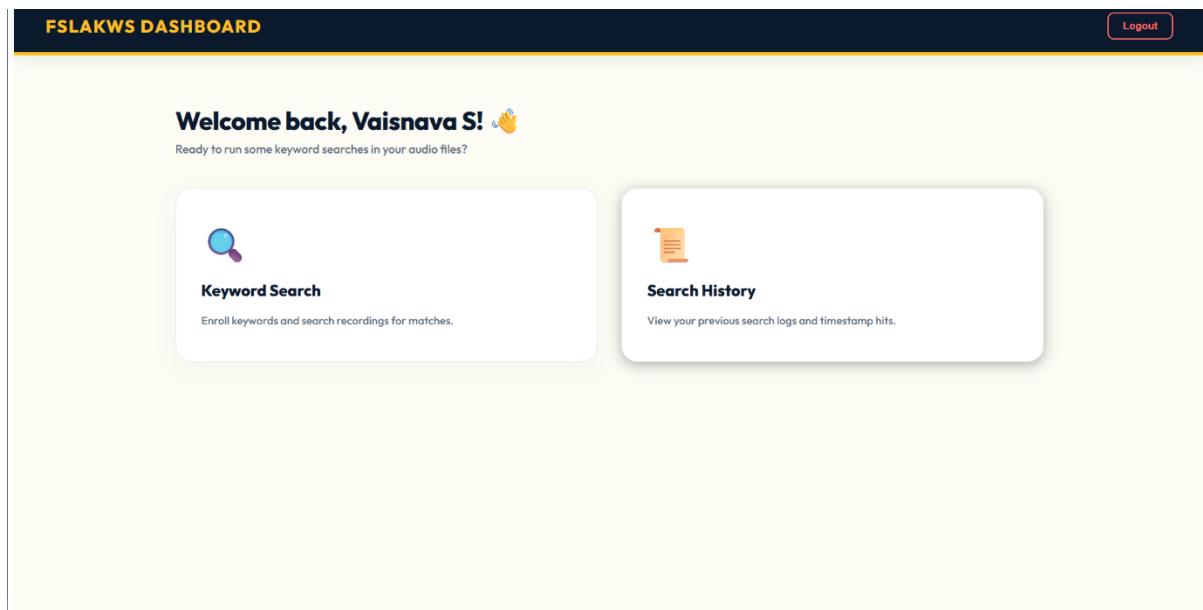
New users who do not have an account can register using the "Create Account" option.

The image shows a registration form titled "CREATE ACCOUNT" with the subtitle "Sign up to scan keywords in audio". The form is centered on a light yellow background. It contains several input fields: "Full Name" with the value "Vaisnava S", "Email Address" with the value "vaisnav@gmail.com", "Password" with four dots, and "Confirm Password" with four dots. Below these is a "Register As" dropdown menu set to "Standard User". At the bottom of the form is a large orange "Register" button. Below the button is a link "Already have an account?" and a "Sign In" button.

*Figure 5. 2 Registration page*

### 5.1.2 Dashboard

This page enables the user to access the functionalities of the system.



*Figure 5. 3 Dashboard*

### 5.1.3 Keyword search pages

This enables the user to upload or record the keyword audio, upload the target audio.

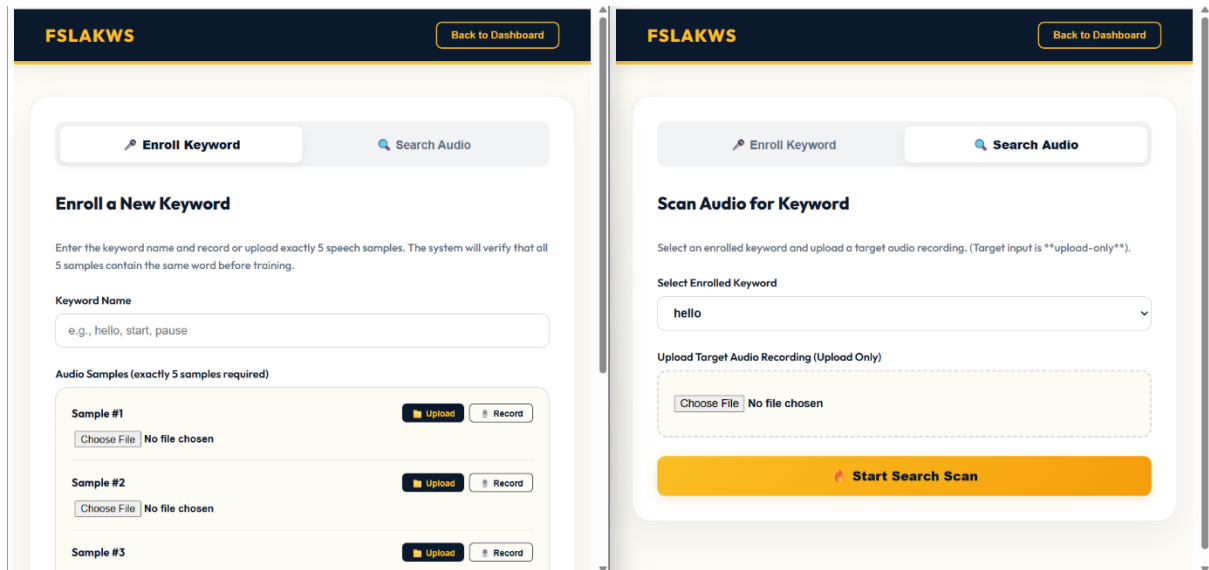
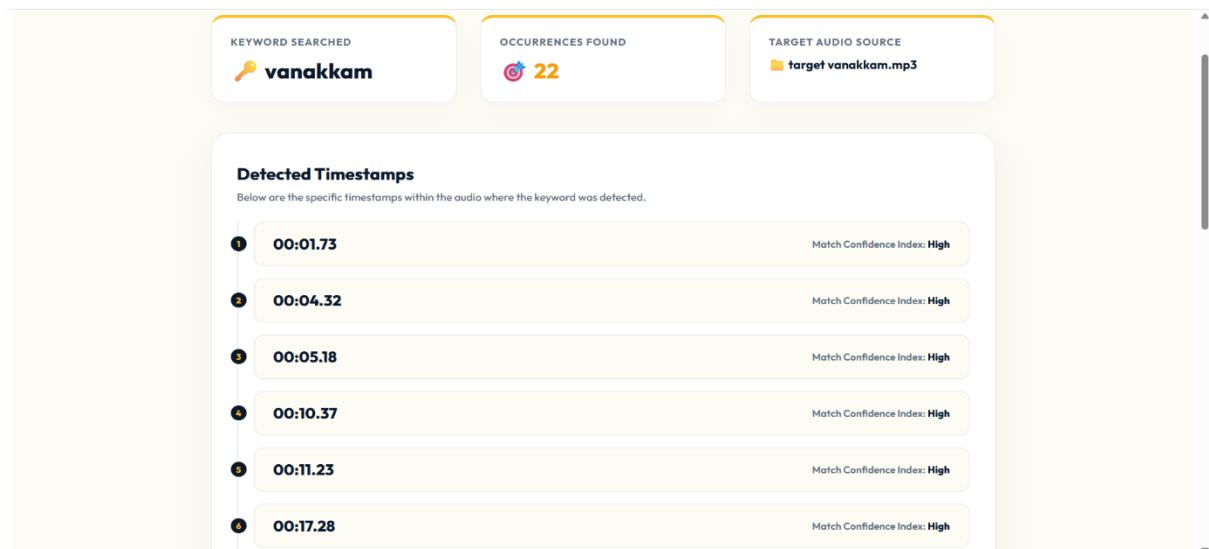


Figure 5. 4 Keyword search pages

### 5.1.4 Result page

This presents the result of keyword search by the user.

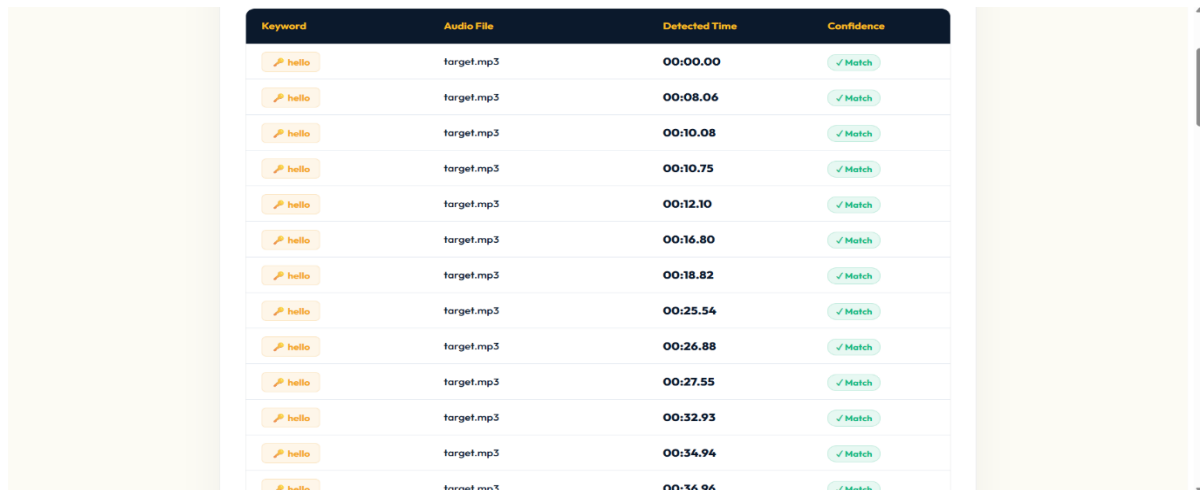


| Timestamp | Match Confidence Index |
|-----------|------------------------|
| 00:01.73  | High                   |
| 00:04.32  | High                   |
| 00:05.18  | High                   |
| 00:10.37  | High                   |
| 00:11.23  | High                   |
| 00:17.28  | High                   |

Figure 5. 5 Result page

### 5.1.5 Search History page

This page maintains a chronological record of keyword spotting operations, allowing users to view details of previous searches, including the keyword, target audio, detection timestamps, and search date.



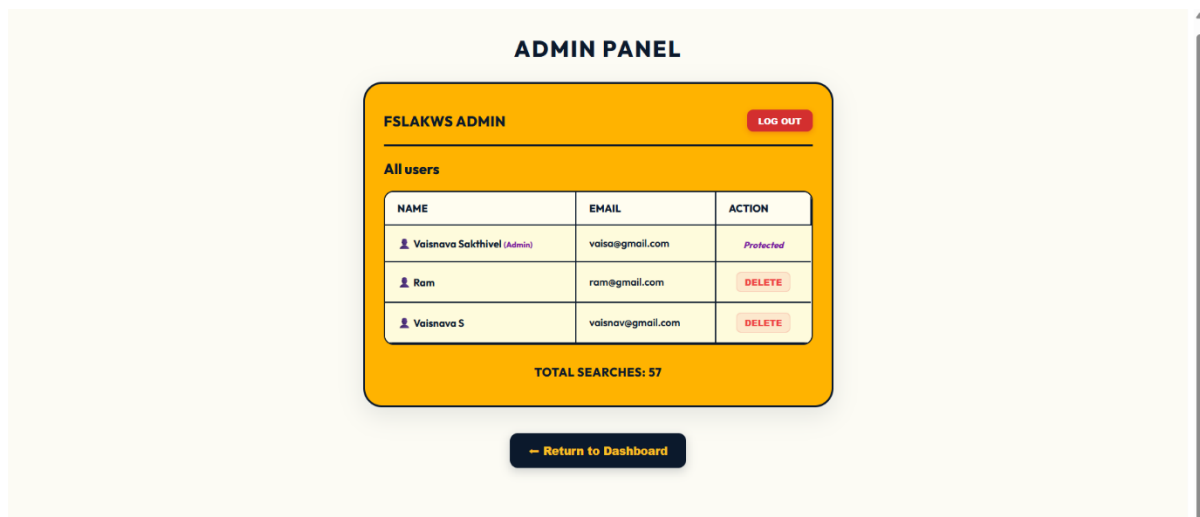
The screenshot shows a table with four columns: Keyword, Audio File, Detected Time, and Confidence. The table contains 13 rows of search results for the keyword 'hello' in the audio file 'target.mp3'. Each row shows a detected time and a 'Match' status.

| Keyword | Audio File | Detected Time | Confidence |
|---------|------------|---------------|------------|
| hello   | target.mp3 | 00:00.00      | ✓ Match    |
| hello   | target.mp3 | 00:08.06      | ✓ Match    |
| hello   | target.mp3 | 00:10.08      | ✓ Match    |
| hello   | target.mp3 | 00:10.75      | ✓ Match    |
| hello   | target.mp3 | 00:12.10      | ✓ Match    |
| hello   | target.mp3 | 00:16.80      | ✓ Match    |
| hello   | target.mp3 | 00:18.82      | ✓ Match    |
| hello   | target.mp3 | 00:25.54      | ✓ Match    |
| hello   | target.mp3 | 00:26.88      | ✓ Match    |
| hello   | target.mp3 | 00:27.55      | ✓ Match    |
| hello   | target.mp3 | 00:32.93      | ✓ Match    |
| hello   | target.mp3 | 00:34.94      | ✓ Match    |
| hello   | target.mp3 | 00:36.96      | ✓ Match    |

Figure 5. 6 Search History page

### 5.1.6 Admin Panel

Admin controls and maintain the actions done by the user.



The screenshot shows the Admin Panel interface. It includes a header 'ADMIN PANEL' and a sub-header 'FSLAKWS ADMIN' with a 'LOG OUT' button. Below this is a section 'All users' containing a table with columns 'NAME', 'EMAIL', and 'ACTION'. The table lists three users: Vaisnava Sakthivel (Admin), Ram, and Vaisnava S. The 'ACTION' column shows 'Protected' for the admin and 'DELETE' for the other two. At the bottom of the panel, it says 'TOTAL SEARCHES: 57' and a 'Return to Dashboard' button.

| NAME                       | EMAIL             | ACTION    |
|----------------------------|-------------------|-----------|
| Vaisnava Sakthivel (Admin) | vaisa@gmail.com   | Protected |
| Ram                        | ram@gmail.com     | DELETE    |
| Vaisnava S                 | vaisnav@gmail.com | DELETE    |

Figure 5. 7 Admin panel

## 5.2 Testing

*Table 5.1 Table of testing and it's results*

| Test Case                      | Expected Result                   | Actual Result                                    | Status           |
|--------------------------------|-----------------------------------|--------------------------------------------------|------------------|
| 5 matching keyword samples     | Verification passes               | Verification passed                              | Pass             |
| 5 non-matching keyword samples | Error displayed                   | Error displayed                                  | Pass             |
| Target audio search            | Timestamps of keyword occurrences | Timestamps returned and spot-checked as accurate | Pass             |
| Upload-only target audio       | No record option shown            | No record option shown                           | Pass             |
| Frontend registration          | User saved to backend             | Initial CORS error, resolved                     | Pass (after fix) |

## 5.3 Observations

- MFCC + DTW-based matching correctly identified keyword occurrences in real recorded audio.
- The speech-recognition-based verification step successfully caught mismatched keyword samples.
- DTW search time increases with the length of the target audio, since the sliding window comparison is computationally intensive.
- Frontend-backend integration required careful handling of CORS configuration.

## 5.4 Advantages

- Requires only five sample recordings, not a large training dataset.
- Language-agnostic, since detection is based on acoustic similarity rather than a trained language model.

- Verifies keyword consistency automatically before performing a search.
- Supports both file upload and live recording for flexibility.
- Modular three-tier architecture allows independent development and testing of the ML service.
- Low-cost, using only open-source libraries and no specialized hardware.

## 5.5 Future Scopes

While FSLAKWS successfully demonstrates a working few-shot, language-agnostic keyword spotting approach, there are several directions in which the system can be extended and improved:

1. **Deep Learning-Based Embeddings** — Replacing MFCC + DTW with neural audio embeddings (e.g., x-vectors, wav2vec 2.0, or speaker/keyword embedding models) to improve detection accuracy and robustness to background noise and speaker variation.
2. **Multi-Keyword Simultaneous Search** — Extending the system to allow enrolment and search of multiple keywords in a single pass over the target audio, rather than one keyword at a time.
3. **Batch Processing of Target Audio** — Enabling users to upload multiple target audio files at once and search all of them for a keyword in a single operation, useful for scanning large recording archives.
4. **Real-Time Keyword Spotting** — Adapting the system to process streaming or live audio input, enabling real-time detection instead of only searching pre-recorded files.
5. **Noise Robustness Improvements** — Incorporating audio preprocessing techniques such as noise reduction, silence trimming, and normalization to improve accuracy on low-quality or noisy recordings.
6. **Mobile Application Support** — Developing a mobile app version of FSLAKWS so users can enrol keywords and search audio directly from a smartphone, expanding accessibility.
7. **Confidence Scoring and Visualization** — Displaying a visual waveform of the target audio with detected keyword regions highlighted, along with confidence scores for each match, to improve interpretability of results.

## 6. CONCLUSION

FSLAKWS successfully demonstrates that accurate keyword detection in audio can be achieved without large labelled datasets, using only five sample recordings and MFCC + DTW-based matching. Its speech-recognition-based verification step ensures reliable keyword templates by confirming all samples match before search begins, making the system language-agnostic and vocabulary-independent.

Implemented as a three-tier application with a React frontend, Spring Boot backend, Python ML service, and MySQL database, the project required real-world integration across multiple technologies. Challenges like audio format handling, DTW tuning, and CORS issues were resolved, and testing on real audio confirmed reliable, accurate detection.

Overall, FSLAKWS proves to be a practical alternative to conventional ASR-based keyword spotting, especially for multilingual or unpredictable-keyword scenarios. With future enhancements like deep learning embeddings and real-time processing, it lays a strong foundation for a production-ready system, while also strengthening core full-stack and problem-solving skills.

## REFERENCES

1. Benesty, J., Sondhi, M. M., and Huang, Y. (2008). *Springer Handbook of Speech Processing*. Springer.
2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. Few-Shot Keyword Spotting in Any Language: <https://arxiv.org/abs/2104.01454>
4. Few-Shot Keyword Spotting from Mixed Speech: <https://arxiv.org/html/2407.06078v1arxiv>
5. Furui, S. (2001). *Digital Speech Processing, Synthesis, and Recognition*. Marcel Dekker.
6. Gold, B., and Morgan, N. (2000). *Speech and Audio Signal Processing: Processing and Perception of Speech and Music*. Wiley.
7. Huang, X., Acero, A., and Hon, H.-W. (2001). *Spoken Language Processing: A Guide to Theory, Algorithm, and System Development*. Prentice Hall.
8. Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
9. Nadas, A. (1985). *Speech Recognition: A Tutorial*. Springer.
10. O'Shaughnessy, D. (2000). *Speech Communications: Human and Machine*. IEEE Press.
11. Rabiner, L. R., and Juang, B. H. (1993). *Fundamentals of Speech Recognition*. Prentice Hall.
12. Raschka, S., Liu, Y., and Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing.
13. Sproat, R. (Ed.). (1998). *Multilingual Text-to-Speech Synthesis: The Bell Labs Approach*. Kluwer Academic Publishers.
14. Taylor, P. (2009). *Text-to-Speech Synthesis*. Cambridge University Press.
15. Wang, Y., Yao, Q., Kwok, J. T., & Ni, L. M. (2020). *Generalizing from a Few Examples: A Survey on Few-Shot Learning*. *ACM Computing Surveys (CSUR)*, **53**(3), 1–34.
16. Xiong, Y., Xu, K., Dou, Y., Zhao, Y., & Gao, Z. (2021). *Wrmatch: Improving FixMatch with Weighted Nuclear-Norm Regularization for Few-Shot Remote Sensing Scene Classification*. *IEEE Transactions on Geoscience and Remote Sensing*, **60**, 1–14.